

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
AI Practical

Practical 3

Roll No.: T043	Name: Hanoch Shetty
Class: TYIT	Batch: 1
Date of Assignment: 18/07/2026	Date/Time of Submission: 18/07/2026

1. Given an initial configuration of the 8-puzzle and a goal configuration, write a Python program to find the solution using Greedy Best-First Search with the Manhattan Distance heuristic.

Code:

```
import heapq

def print_state(state):
    for i in range(0,9,3):
        print(state[i],state[i+1],state[i+2])
    print()

def heuristic(state,goal):
    distance = 0
    for num in range(1,9):
        current = state.index(num)
        target = goal.index(num)
        x1, y1 = current // 3, current % 3
        x2, y2 = target // 3, target % 3
        distance += abs(x1 - x2) + abs(y1 - y2)
    return distance

def get_neighbors(state):
    neighbors = []
    blank = state.index(0)
    moves = [-3, 3, -1, 1]
    for move in moves:
        new = blank + move
        if new >= 0 and new <9:
            if blank %3==0 and move==-1:
                continue
            if blank %3==2 and move==1:
```

```

        continue
    temp = list(state)
    temp[blank], temp[new] = temp[new], temp[blank]
    neighbors.append(tuple(temp))
return neighbors
def greedy(initial, goal):
    queue = []
    heapq.heappush(
        queue,
        (heuristic(initial,goal), initial, []))
    visited = set()
    while queue:
        h,state,path = heapq.heappop(queue)
        if state == goal:
            return path + [state]
        if state not in visited:
            visited.add(state)
            for next_state in get_neighbors(state):
                heapq.heappush(
                    queue,
                    (heuristic(next_state, goal),
                     next_state,
                     path + [state]))

    return None
initial = (1,2,3,4,0,6,7,5,8)
goal = (1,2,3,4,5,6,7,8,0)
print("Initial State: ")
print_state(initial)
print("Goal State: ")
print_state(goal)
solution = greedy(initial,goal)
print("Solution: ")
for step, state in enumerate(solution):
    print("Step",step)
    print_state(state)
print("Goal Reached Successfully.")

```

Output:

Initial State:

```

1 2 3
4 0 6
7 5 8

```

Goal State:

```

1 2 3
4 5 6
7 8 0

```

Solution:

Step 0

```

1 2 3
4 0 6
7 5 8

```

Step 1

```

1 2 3
4 5 6
7 0 8

```

Step 2

```

1 2 3
4 5 6
7 8 0

```

Goal Reached Successfully.

HANOCH SHETTY T043

2. Given an initial configuration of the 8-puzzle and a goal configuration, write a Python program to find the shortest path using A* search with the Manhattan Distance heuristic.

Code:

```
def print_state(state):
    for i in range(0,9,3):
        print(state[i],state[i+1],state[i+2])
    print()

def heuristic(state,goal):
    distance = 0
    for num in range(1,9):
        current = state.index(num)
        target = goal.index(num)
        x1, y1 = current //3, current %3
        x2, y2 = target //3, target % 3
        distance += abs(x1 - x2) + abs(y1 - y2)
    return distance

def get_neighbors(state):
    neighbors = []
    blank = state.index(0)
    moves = [-3, 3, -1, 1]
    for move in moves:
        new = blank + move
        if new >= 0 and new <9:
            if blank %3==0 and move== -1: continue
            if blank %3==2 and move== 1: continue
            temp = list(state)
            temp[blank], temp[new] = temp[new], temp[blank]
            neighbors.append(tuple(temp))
    return neighbors

def astar(initial, goal):
    queue = []
    heapq.heappush(queue, (heuristic(initial,goal), 0,initial, []))
    visited = set()
    while queue:
        f, g, state, path = heapq.heappop(queue)
        if state == goal:
            return path + [state]
        if state not in visited:
            visited.add(state)
            for next_state in get_neighbors(state):
                new_g = g + 1
                new_f = new_g + heuristic(next_state, goal)
                heapq.heappush(
                    queue,
                    (new_f,new_g, next_state, path+[state]))
    return None

initial = (1,2,3,4,0,6,7,5,8)
goal = (1,2,3,4,5,6,7,8,0)
print("Initial State: ")
print_state(initial)
print("Goal State: ")
```

```

print_state(goal)
solution = astar(initial,goal)
print("Shortest Path using A* Search: ")
step = 1
for state in solution[1:]:
    print("Step",step)
    print_state(state)
    step+=1
print("Goal reached Successfully.")
print("Total Number of moves: ",len(solution)-1)

```

Output:

```

Initial State:
1 2 3
4 0 6
7 5 8

Goal State:
1 2 3
4 5 6
7 8 0

Shortest Path using A* Search:
Step 1
1 2 3
4 5 6
7 0 8

Step 2
1 2 3
4 5 6
7 8 0

Goal reached Successfully.
Total Number of moves:  2
HANOCH SHETTY T043

```

3. Given a weighted graph representing cities and distances between them, write a Python program to find the shortest path from Arad to Bucharest using A* search with a heuristic function (straight-line distance to destination).

Code:

```
graph = {
    'Mumbai': [ ('Pune', 75), ('Indore', 500), ('Delhi', 700) ],
    'Pune': [ ('Mumbai', 75), ('Indore', 300), ('Delhi', 500) ],
    'Indore': [ ('Mumbai', 500), ('Pune', 300), ('Delhi', 200) ],
    'Delhi': [ ('Mumbai', 700), ('Pune', 500), ('Indore', 200) ],
    'Navi Mumbai': [] }
heuristic = { 'Mumbai': 20,
    'Pune': 100,
    'Delhi': 650,
    'Indore': 450,
    'Navi Mumbai': 0, }
def astar(start, goal):
    queue = []
    heapq.heappush(queue, (heuristic[start], 0, start, [start]))
    visited = set()
    while queue:
        f_cost, g_cost, city, path = heapq.heappop(queue)
        if city == goal: return g_cost, path
        if city not in visited: visited.add(city)
        for neighbor, distance in graph[city]:
            new_g = g_cost + distance
            new_f = new_g + heuristic[neighbor]
            heapq.heappush(queue, (new_f, new_g, neighbor, path+[neighbor]))
    return None, []
cost, path = astar('Mumbai', 'Delhi')
print("Shortest Path using A* Search: ")
print("Path: ", "->".join(path))
print("Total Cost: ", cost)
```

Output:

```
Shortest Path using A* Search:
Path:  Mumbai->Pune->Delhi
Total Cost:  575
HANOCH SHETTY T043
```